```
!nvidia-smi
!nvcc --version
!pip install nvcc4jupyter
%load_ext nvcc4jupyter

%%cuda
#include <iostream>
#include <cuda_runtime.h>

using namespace std;

// CUDA Kernel for Matrix Multiplication
__global__ void matrixMul(const int* a, const int* b, int* c, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < n && col < n) {
        int sum = 0;
        for (int k = 0; k < n; k++) {
            sum += a[row * n + k] * b[k * n + col];
        }
        c[row * n + col] = sum;
    }
}

int main() {
    int n = 2; // Size of matrix (n x n)
    size_t size = n * n * sizeof(int);

    int *h_a = new int[n * n];
    int *h_b = new int[n * n];
```

```
int *h_c = new int[n * n];

// Hardcoded values for 2x2 Matrix A and B
// A = [1 2; 3 4], B = [5 6; 7 8]
int init_a[] = {1, 2, 3, 4};
int init_b[] = {5, 6, 7, 8};

for (int i = 0; i < n * n; i++) {
    h_a[i] = init_a[i];
    h_b[i] = init_b[i];
}

cout << "Matrix A:" << endl;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) cout << h_a[i * n + j] << " ";
    cout << endl;
}

cout << "Matrix B:" << endl;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) cout << h_b[i * n + j] << " ";
    cout << endl;
}

int *d_a, *d_b, *d_c;
cudaMalloc(&d_a, size);
cudaMalloc(&d_b, size);
cudaMalloc(&d_c, size);

cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);
```

```
    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((n + 15) / 16, (n + 15) / 16);

    matrixMul<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    cout << "Result Matrix C:" << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) cout << h_c[i * n + j] << " ";
        cout << endl;
    }

    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    delete[] h_a; delete[] h_b; delete[] h_c;

    return 0;
}
```